User Model

```
from django.db import models
from django.contrib.auth.models import AbstractUser
from users.managers import CustomUserManager
# Create your models here.

class User(AbstractUser):
    username=None
    email = models.EmailField(unique=True)
    address = models.TextField(blank=True,null=True)
    phone_number = models.CharField(max_length=15,blank=True, null=True)

    USERNAME_FIELD = 'email' # use email instead of uesername
    REQUIRED_FIELDS = []

    objects = CustomUserManager()

    def __str__(self):
        return self.email
```

তোমার কোডটি **Django Custom User Model**। এখানে তুমি Django-র default `username` login বাদ দিয়ে **email দিয়ে login system** তৈরি করছ। নিচে পুরো code **line by line explain** করছি।

1️⃣ Import Section

```
from django.db import models
```

👉 Django ORM এর model class ব্যবহার করার জন্য।

Example:

```
models.CharField
models.EmailField
models.TextField
```

```
from django.contrib.auth.models import AbstractUser
```

👉 Django এর **default user model base class**।

`AbstractUser` এর মধ্যে already অনেক field থাকে:

Field	কাজ
username	username
first_name	first name
last_name	last name
email	email
password	password
is_staff	admin access
is_superuser	full permission
is_active	active user

তুমি এই model **extend** করে নতুন field যোগ করছ।

```
from users.managers import CustomUserManager
```

👉 এটা তোমার আগের বানানো **CustomUserManager**।

এর কাজ:

```
create_user()
create_superuser()
```

define করা।

2️⃣ User Model

```
class User(AbstractUser):
```

👉 তুমি **Custom User Model** তৈরি করছ।

মানে:

```
Django default user
        +
extra fields
```

```
        +
email login
```

3️⃣ Remove username

```
username = None
```

👉 Django default username field remove করছ।

Default Django login:

```
username + password
```

তুমি পরিবর্তন করেছ:

```
email + password
```

4️⃣ Email Field

```
email = models.EmailField(unique=True)
```

👉 Email field তৈরি করা হয়েছে।

Important:

```
unique=True
```

মানে একই email দুইবার register করা যাবে না।

Example ❌

```
test@gmail.com
test@gmail.com
```

Database error দিবে।

5️⃣ Address Field

```
address = models.TextField(blank=True, null=True)
```

👉 User address।

Explanation:

Parameter	Meaning
blank=True	form এ optional
null=True	database এ null allowed

Example:

```
address = ""
or
address = NULL
```

6️⃣ Phone Number

```
phone_number = models.CharField(max_length=15,blank=True, null=True)
```

👉 User phone number।

Why `CharField`?

কারণ phone number কখনো number type না।

Example:

```
+8801712345678
01712345678
```

7️⃣ USERNAME_FIELD

```
USERNAME_FIELD = 'email'
```

👉 Django কে বলা হচ্ছে:

```
login field = email
```

Default:

```
USERNAME_FIELD = "username"
```

তুমি change করেছ।

Login example:

```
email: test@gmail.com
password: ********
```

8️⃣ REQUIRED_FIELDS

```
REQUIRED_FIELDS = []
```

👉 `createsuperuser` command এ কোন extra field লাগবে।

Default Django:

```
username
email
```

তোমার ক্ষেত্রে:

```
only email
```

Example:

```
python manage.py createsuperuser
```

ask করবে:

```
email
password
```

9️⃣ Custom Manager

```
objects = CustomUserManager()
```

👉 Django ORM কে বলা হচ্ছে:

```
user creation logic → CustomUserManager
```

Example:

```
User.objects.create_user()
User.objects.create_superuser()
```

এগুলো তোমার manager এর method run করবে।

1️⃣0️⃣ String Representation

```
def __str__(self):
    return self.email
```

👉 Model কে string হিসেবে represent করে।

Admin panel বা shell এ দেখাবে।

Example

```
<User: test@gmail.com>
```

instead of

```
<User: User object (1)>
```

📊 Database Table

এই model run করলে database table হবে।

Example

```
users_user
```

Fields:

Field
id
email
password
first_name
last_name
address
phone_number
is_staff
is_superuser

⚠️ Important Settings (Must Do)

settings.py তে দিতে হবে:

```
AUTH_USER_MODEL = "users.User"
```

না দিলে Django default user use করবে।

🚀 Full Flow

```
User registration
        |
        ▼
CustomUserManager.create_user()
        |
        ▼
password hashing
        |
        ▼
save user
        |
        ▼
login using email
```

💡 Example API Data

User register request:

```
{
 "email":"atiar@gmail.com",
 "password":"123456",
 "phone_number":"01700000000"
}
```

Database:

```
email = atiar@gmail.com
password = hashed
phone_number = 01700000000
```

⭐ Small Improvement (Professional)

Production এ সাধারণত phone number এ validation দেয়।

Example:

```
phone_number = models.CharField(
    max_length=15,
    blank=True,
    null=True,
    help_text="Enter phone number"
)
```

✅ চাইলে আমি তোমাকে দেখাতে পারি:

- **Django Custom User + JWT Authentication**
- **User Register API**
- **Login API**
- **Refresh Token**
- **Role Based Authentication**

যেটা **DRF production backend এ standard practice**।

> **User register করলে default role = USER হবে, কিন্তু admin পরে তাকে ADMIN permission দিতে পারবে।**

এটা Django-তে খুব common pattern। নিচে **clean solution** দেখাচ্ছি।

1️⃣ User Model (Default USER Role)

```
from django.db import models
from django.contrib.auth.models import AbstractUser
from users.managers import CustomUserManager


class User(AbstractUser):

    username = None

    email = models.EmailField(unique=True)
    address = models.TextField(blank=True, null=True)
    phone_number = models.CharField(max_length=15, blank=True, null=True)

    # Roles
    ADMIN = "ADMIN"
    STAFF = "STAFF"
    USER = "USER"

    ROLE_CHOICES = [
        (ADMIN, "Admin"),
        (STAFF, "Staff"),
        (USER, "User"),
    ]

    role = models.CharField(
        max_length=10,
        choices=ROLE_CHOICES,
        default=USER   # default role
    )

    USERNAME_FIELD = "email"
    REQUIRED_FIELDS = []

    objects = CustomUserManager()

    def __str__(self):
        return self.email
```

Important

```
default=USER
```

মানে:

User register করলে automatically role হবে

```
USER
```

User role select করতে পারবে না।

2️⃣ Admin Later Change Role

Admin panel থেকে role change করা যাবে।

Example

```
USER → STAFF
USER → ADMIN
```

3️⃣ Admin Panel Setup

`admin.py`

```python
from django.contrib import admin
from .models import User


@admin.register(User)
class UserAdmin(admin.ModelAdmin):

    list_display = ["email", "role", "is_staff"]

    list_filter = ["role"]

    search_fields = ["email"]
```

Admin panel এ role dropdown দেখাবে।

4️⃣ Example Flow

Step 1: User Registration

API request

```
{
 "email": "user@gmail.com",
 "password": "123456"
}
```

Database:

email	role
user@gmail.com	USER

Step 2: Admin Change Role

Admin panel

```
USER → STAFF
```

Database:

email	role
user@gmail.com	STAFF

5️⃣ Optional: Auto Set Django Admin Flags

যদি role = ADMIN হয় তাহলে automatically

```
is_staff=True
is_superuser=True
```

করতে পারো।

Example:

```
def save(self, *args, **kwargs):
```

```
    if self.role == self.ADMIN:
        self.is_staff = True
        self.is_superuser = True

    super().save(*args, **kwargs)
```

6️⃣ Professional Production Pattern

Enterprise systems usually follow:

```
User Register
      ↓
Default Role = USER
      ↓
Admin Promote
      ↓
STAFF / MANAGER / ADMIN
```

🚀 Pro Tip

তুমি যেহেতু **Django + DRF backend** বানাচ্ছো, best practice হলো:

```
User Register → role = USER
Admin API → change role
```

```
from django.contrib.auth.base_user import BaseUserManager

class CustomUserManager(BaseUserManager):

    def create_user(self, email, password=None, **extra_fields):

        if not email:
            raise ValueError("Email must be provided")

        email = self.normalize_email(email)

        user = self.model(email=email, **extra_fields)

        user.set_password(password)
```

```
        user.save(using=self._db)

        return user


    def create_superuser(self, email, password=None, **extra_fields):

        extra_fields.setdefault("is_staff", True)
        extra_fields.setdefault("is_superuser", True)
        extra_fields.setdefault("is_active", True)

        if extra_fields.get("is_staff") is not True:
            raise ValueError("Superuser must have is_staff=True")

        if extra_fields.get("is_superuser") is not True:
            raise ValueError("Superuser must have is_superuser=True")

        return self.create_user(email, password, **extra_fields)
```

এই কোডটি Django-তে **Custom User Model এর Manager**।
এর কাজ হলো **user এবং superuser কিভাবে create হবে তা define করা**।

যখন তুমি run করো:

```
python manage.py createsuperuser
```

অথবা

```
User.objects.create_user()
```

তখন এই **CustomUserManager এর method গুলো run হয়**।

এখন **line by line explain** করি।

1️⃣ Import

```
from django.contrib.auth.base_user import BaseUserManager
```

`BaseUserManager` হলো Django-র **base class for user managers**।

এটা provide করে useful function যেমন:

- `normalize_email()`
- user creation helpers

তুমি এটাকে inherit করে **Custom manager** বানাচ্ছ।

2️⃣ CustomUserManager Class

```
class CustomUserManager(BaseUserManager):
```

এখানে তুমি **User model এর manager define করছ**।

Manager এর কাজ:

```
Database object create / query
```

Example:

```
User.objects.create_user()
User.objects.create_superuser()
```

এগুলো এখানেই define করা।

3️⃣ create_user()

```
def create_user(self, email, password=None, **extra_fields):
```

এটা **normal user create করার method**।

Parameters:

Parameter	Meaning
email	user email
password	user password
extra_fields	extra fields (phone, address etc.)

Example call:

```
User.objects.create_user(
    email="test@gmail.com",
    password="123456",
```

```
    phone_number="01700000000"
)
```

4️⃣ Email Validation

```
if not email:
    raise ValueError("Email must be provided")
```

যদি email না দেওয়া হয়:

```
error throw করবে
```

Example ❌

```
User.objects.create_user(password="123456")
```

Output

```
ValueError: Email must be provided
```

5️⃣ Normalize Email

```
email = self.normalize_email(email)
```

Email কে **standard format** এ convert করে।

Example

```
TEST@GMAIL.COM
```

becomes

```
test@gmail.com
```

এতে database clean থাকে।

6️⃣ Create User Object

```
user = self.model(email=email, **extra_fields)
```

`self.model` = **User model**

Example internally:

```
user = User(
    email=email,
    phone_number="01700000000"
)
```

এখানে object create হয় কিন্তু database এ save হয়নি।

7️⃣ Password Hashing

```
user.set_password(password)
```

এটা খুব important।

Password **plain text store হয় না**।

Example input:

```
123456
```

Database এ stored:

```
pbkdf2_sha256$720000$......
```

এটাকে বলে **password hashing**।

Security purpose।

8️⃣ Save User

```
user.save(using=self._db)
```

User object database এ save হয়।

`self._db` মানে:

```
current database connection
```

Multi-database project এ useful।

9️⃣ Return User

```
return user
```

শেষে created user object return করে।

Example output:

```
<User: test@gmail.com>
```

🔟 create_superuser()

```
def create_superuser(self, email, password=None, **extra_fields):
```

এই method **admin user create করার জন্য**।

Run হয় যখন:

```
python manage.py createsuperuser
```

1️⃣1️⃣ Default Admin Permissions

```
extra_fields.setdefault("is_staff", True)
extra_fields.setdefault("is_superuser", True)
extra_fields.setdefault("is_active", True)
```

এগুলো automatically set হয়।

Field	Meaning
is_staff	admin panel access

Field	Meaning
is_superuser	full permission
is_active	account active

Example superuser:

```
is_staff = True
is_superuser = True
```

1️⃣2️⃣ Validation Check

```
if extra_fields.get("is_staff") is not True:
    raise ValueError("Superuser must have is_staff=True")
```

যদি কেউ ভুল করে:

```
create_superuser(is_staff=False)
```

তাহলে error দিবে।

1️⃣3️⃣ Superuser Permission Check

```
if extra_fields.get("is_superuser") is not True:
    raise ValueError("Superuser must have is_superuser=True")
```

Superuser অবশ্যই:

```
is_superuser=True
```

হতে হবে।

1️⃣4️⃣ Create Superuser

```
return self.create_user(email, password, **extra_fields)
```

এখানে smart trick ব্যবহার হয়েছে।

Superuser আসলে:

```
normal user
+ extra permissions
```

তাই `create_user()` reuse করা হয়েছে।

📊 Full Flow Diagram

```
create_superuser()
        |
        ▼
set admin permissions
        |
        ▼
call create_user()
        |
        ▼
normalize email
        |
        ▼
hash password
        |
        ▼
save user
```

🔑 Example Usage

Create normal user

```
User.objects.create_user(
    email="user@gmail.com",
    password="123456"
)
```

Create superuser

```
python manage.py createsuperuser
```

Terminal

```
Email: admin@gmail.com
Password:
```

⚠️ Important

এই manager কাজ করবে যদি model এ থাকে:

```
objects = CustomUserManager()
```

Example

```
class User(AbstractUser):

    username = None
    email = models.EmailField(unique=True)

    objects = CustomUserManager()
```

🚀 Pro Tip

Professional Django projects এ **CustomUserManager + CustomUserModel** ব্যবহার করা হয় কারণ:

```
username login → outdated
email login → modern standard
```

```
from django.contrib import admin
from users.models import User
from django.contrib.auth.admin import UserAdmin


# Register your models here.
class CustomUserAdmin(UserAdmin):
    model = User
```

```
    list_display = ('email','first_name','last_name','is_active')
    list_filter =('is_staff','is_active')

    fieldsets = (
        (None,{'fields':('email','password')}),
        ('Personal Info',{'fields':
('first_name','last_name','address','phone_number')}),
        ('Permissions', {'fields':
('is_staff','is_active','is_superuser','groups','user_permissions')}),
        ('Important Dates',{'fields':('last_login','date_joined')})
    )

    add_fieldsets=(
        (None,{
            'classes':('wide',),
            'fields':
('email','password1','password2','is_staff','is_active')
        })
    )

    search_fields = ('email',)
    ordering = ('email',)


admin.site.register(User,CustomUserAdmin)
```

তোমার কোডটি **Django Admin Panel-এ Custom User Model কিভাবে দেখাবে ও manage করবে** তা configure করছে। কারণ তুমি `AbstractUser` modify করে **custom User model** বানিয়েছ, তাই admin-এ সেটাকে customize করতে হয়।

আমি **line-by-line explain** করছি।

1️⃣ Import Section

```
from django.contrib import admin
```

এটা Django-র **admin framework** import করে।

Admin panel URL:

```
http://127.0.0.1:8000/admin
```

```
from users.models import User
```

এখানে তোমার **custom User model** import করা হয়েছে।

তুমি default `auth.User` ব্যবহার করছ না।

```
from django.contrib.auth.admin import UserAdmin
```

`UserAdmin` হলো Django-র built-in **admin configuration for user models**।

এটা provide করে:

- password hashing
- permissions
- groups
- user permissions
- user creation form

তুমি এটাকে **extend** করছ।

2️⃣ CustomUserAdmin Class

```
class CustomUserAdmin(UserAdmin):
```

মানে:

```
Default Django UserAdmin
        +
Custom configuration
```

3️⃣ Model Define

```
model = User
```

Admin কে বলা হচ্ছে:

```
এই admin class → User model manage করবে
```

4️⃣ list_display

```
list_display = ('email','first_name','last_name','is_active')
```

Admin panel এর **user list page** এ কোন column দেখাবে।

Example table:

Email	First Name	Last Name	Active
user@gmail.com	Atiar	Rahman	True

5️⃣ list_filter

```
list_filter = ('is_staff','is_active')
```

Admin panel এর **right sidebar filter**।

Example:

```
Filter
------
is_staff
True
False

is_active
True
False
```

Admin easily filter করতে পারে।

6️⃣ fieldsets

```
fieldsets = (
```

এটা define করে **admin user edit page layout**।

Admin panel এ field গুলো group হয়ে দেখাবে।

Section 1

```
(None,{'fields':('email','password')}),
```

Section name নেই (`None`)

Fields:

```
email
password
```

Section 2

```
('Personal Info',{'fields':
('first_name','last_name','address','phone_number')}),
```

Admin page এ section:

```
Personal Info
-------------
first_name
last_name
address
phone_number
```

Section 3

```
('Permissions', {'fields':
('is_staff','is_active','is_superuser','groups','user_permissions')}),
```

User permissions manage করার section।

Fields:

Field	Meaning
is_staff	admin panel access
is_active	account active
is_superuser	full permission
groups	permission groups
user_permissions	custom permission

Section 4

```
('Important Dates',{'fields':('last_login','date_joined')})
```

User activity information।

Field	Meaning
last_login	last login time
date_joined	registration date

7️⃣ add_fieldsets

```
add_fieldsets=(
```

এটা **new user create form** layout define করে।

Admin যখন **Add User** click করবে তখন এই fields দেখাবে।

```
(None,{
    'classes':('wide',),
```

`wide` = admin form একটু wide layout হবে।

```
'fields':('email','password1','password2','is_staff','is_active')
```

Admin create user form:

Field
email
password
confirm password
is_staff
is_active

8️⃣ search_fields

```
search_fields = ('email',)
```

Admin search bar।

Admin search করতে পারবে:

```
user@gmail.com
```

9️⃣ ordering

```
ordering = ('email',)
```

User list **email অনুযায়ী sort হবে**।

Example:

```
a@gmail.com
b@gmail.com
c@gmail.com
```

🔟 Register Model

```
admin.site.register(User,CustomUserAdmin)
```

এটা Django কে বলে:

```
User model → CustomUserAdmin দিয়ে register করো
```

Result:

Admin panel এ **Users management UI** পাওয়া যাবে।

📊 Admin Panel Structure

Admin UI দেখতে এমন হবে:

```
Users
 ├── Email
 ├── First Name
 ├── Last Name
 ├── Active
```

User edit page:

```
Email
Password

Personal Info
-------------
First Name
Last Name
Address
Phone Number

Permissions
-----------
Is Staff
Is Active
Is Superuser
Groups

Important Dates
---------------
Last Login
Date Joined
```

⚠️ Important

এটা কাজ করবে যদি settings.py তে থাকে:

```
AUTH_USER_MODEL = "users.User"
```

🚀 Small Improvement (Professional)

যদি role field থাকে তাহলে add করতে পারো:

```
list_display = ('email','role','first_name','is_active')
```

এবং

```
('Permissions', {'fields':
('role','is_staff','is_active','is_superuser')}),
```

Review my code

✅ যেগুলো ঠিক আছে

1️⃣ CustomUserAdmin inherit করা

```
class CustomUserAdmin(UserAdmin):
```

✔ ঠিক আছে
এতে Django-র built-in user admin features ব্যবহার করা যায়।

2️⃣ Model define

```
model = User
```

✔ ঠিক আছে
এটা admin কে বলে কোন model manage করবে।

3️⃣ list_display

```
list_display = ('email','first_name','last_name','is_active')
```

✔ ঠিক আছে
Admin list page এ column দেখাবে।

Example:

| Email | First Name | Last Name | Active |

4️⃣ list_filter

```
list_filter =('is_staff','is_active')
```

✔ ঠিক আছে
Admin filter sidebar।

5️⃣ fieldsets

```
fieldsets = (
```

✔ খুব ভালো structure

Admin page section wise organize হবে।

Example:

```
Email
Password

Personal Info
Permissions
Important Dates
```

6️⃣ add_fieldsets

```
add_fieldsets=(
```

✔ ঠিক আছে
Admin panel এ **user create form** customize করে।

7️⃣ search_fields

```
search_fields = ('email',)
```

✔ ভালো practice

Admin search করতে পারবে।

8️⃣ ordering

```
ordering = ('email',)
```

✔ ঠিক আছে

User list email অনুযায়ী sort হবে।

9️⃣ Model register

```
admin.site.register(User,CustomUserAdmin)
```

✔ ঠিক আছে

⚠️ Small Improvements (Recommended)

1️⃣ role field থাকলে add করো

যদি তোমার model এ role থাকে:

```
role = models.CharField(...)
```

তাহলে admin এ দেখানো উচিত।

Better

```
list_display = ('email','first_name','last_name','role','is_active')
```

2️⃣ filter এ role add করা

```
list_filter = ('role','is_staff','is_active')
```

3️⃣ search_fields improvement

```
search_fields = ('email','first_name','last_name')
```

Admin search powerful হবে।

4️⃣ ordering improvement

Better:

```
ordering = ('-date_joined',)
```

Newest user first।

⭐ Improved Version

```
from django.contrib import admin
from users.models import User
from django.contrib.auth.admin import UserAdmin


class CustomUserAdmin(UserAdmin):

    model = User

    list_display = ('email','first_name','last_name','is_active')
    list_filter = ('is_staff','is_active')

    fieldsets = (
        (None, {'fields': ('email','password')}),
```

```
        ('Personal Info', {
            'fields': ('first_name','last_name','address','phone_number')
        }),

        ('Permissions', {
            'fields':
('is_staff','is_active','is_superuser','groups','user_permissions')
        }),

        ('Important Dates', {
            'fields': ('last_login','date_joined')
        }),
    )

    add_fieldsets = (
        (None, {
            'classes': ('wide',),
            'fields':
('email','password1','password2','is_staff','is_active')
        }),
    )

    search_fields = ('email','first_name','last_name')
    ordering = ('email',)


admin.site.register(User, CustomUserAdmin)
```

⭐ Final Verdict

✔ তোমার কোড **correct**
✔ Django best practice follow করছে
✔ Custom user admin properly configure করা হয়েছে

2️⃣ settings.py এ AUTH_USER_MODEL সেট করা

`# settings.py AUTH_USER_MODEL = 'accounts.User'`

Format:

`'app_name.ModelName'`

উদাহরণ:

`accounts.User`